University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik

CRT-Royale [MSL]

Portierung von CRT-Royale in die Metal Shading Language
sowie Implementierung in RetroVisor

Projektdokumentation

Software-Projekt
Sommersemester 2026

Betreuer: Prof. Dr. Hoffmann

Bearbeiter: Devin Uyan

Repository: https://github.com/snowIsNotAvaiable/crt-royale-msl

Version 1.0 – Stand: 4. April 2026

Inhaltsverzeichnis

1 Einleitung . . . **2**

1.1 Motivation . . . 2
1.2 Projektziel . . . 2
1.3 Abgrenzung . . . 2

2 Grundlagen . . . **3**

2.1 GPU-Programmierung und Compute Shader . . . 3
2.2 Metal Shading Language (MSL) . . . 3
2.3 GLSL vs. MSL – Wesentliche Unterschiede . . . 4

3 Ist-Analyse: CRT-Royale . . . **4**

3.1 Überblick . . . 4
3.2 Die 13-Pass-Pipeline . . . 5
3.3 Quellstruktur . . . 5
3.4 Konfigurierbare Parameter (Auszug) . . . 6

4 Zielarchitektur: RetroVisor . . . **6**

4.1 Überblick . . . 6
4.2 Shader-Integrationsmuster . . . 7
4.3 Render-Pipeline . . . 7

5 Technologieauswahl . . . **8**

6 Portierungsstrategie . . . **8**

6.1 Vorgehen . . . 8
6.2 Übersetzungsregeln . . . 8
6.3 Herausforderungen . . . 9

7 Implementierung . . . **9**

7.1 Aktueller Stand . . . 9
7.2 Pass 1: Code-Beispiel (MSL) . . . 10
7.3 Swift-Integration . . . 10
7.4 Dateien im Projekt . . . 11

8 Validierungskonzept . . . **11**

8.1 Methodik . . . 11
8.2 Testmuster . . . 11

9 Evaluierungsplan . . . **12**

9.1 Qualitätsvergleich . . . 12
9.2 Performancebewertung . . . 12

10 Projektplanung . . . **12**

10.1 Phasenübersicht . . . 12
10.2 Detailplan Phase 1: Portierung . . . 12

11 Muss- und Kann-Kriterien . . . **13**

12 Glossar . . . **14**

1 Einleitung

1.1 Motivation

Retro-Gaming erfreut sich wachsender Beliebtheit. Emulatoren wie RetroArch oder OpenEmu ermöglichen es, klassische Konsolenspiele auf moderner Hardware zu spielen. Dabei geht jedoch ein wesentliches visuelles Merkmal verloren: das charakteristische Bild eines **CRT-Monitors** (Cathode Ray Tube) mit seinen Scanlines, dem Phosphor-Glow und der typischen Farbwiedergabe.

CRT-Royale von TroggleMonkey ist einer der aufwendigsten und physikalisch genauesten CRT-Shader. Er simuliert in einer **13-Pass-Pipeline** nahezu alle optischen Eigenschaften eines CRT-Bildschirms – von der Gamma-Korrektur über Phosphormasken bis hin zu Bloom und Geometrieverzerrung. Allerdings existiert CRT-Royale bisher nur für Slang/GLSL (libretro), Cg und HLSL (ReShade). Eine native Portierung auf **Apple Metal (MSL)** fehlt vollständig.

1.2 Projektziel

Ziel dieses Semesterprojekts ist die vollständige Portierung des CRT-Royale-Shaders von Slang/GLSL in die **Metal Shading Language (MSL)** und dessen Integration in **RetroVisor**, eine macOS-Anwendung von Prof. Dr. Hoffmann, die GPU-basierte Bildeffekte als Echtzeit-Overlay auf beliebige Fenster anwendet.

Das Projekt gliedert sich in vier Phasen:

1. **Portierung:** Übersetzung aller 13 Shader-Passes von GLSL nach MSL
2. **Integration:** Einbindung der MSL-Shader in die RetroVisor-Architektur
3. **Validierung:** Visueller Vergleich mit dem GLSL-Original (Comparison-Images)
4. **Evaluierung:** Qualitäts- und Performancebewertung gegenüber dem Original

1.3 Abgrenzung

- Es wird **kein eigener Emulator** entwickelt – RetroVisor ist eine bestehende Anwendung.
- Die Portierung beschränkt sich auf den **CRT-Royale-Shader**.
- Zielplattform ist ausschließlich **macOS mit Apple Silicon** (Metal API).
- Machine-Learning-basierte Upscaling-Verfahren (z. B. FSR, DLSS) sind nicht Gegenstand.

2 Grundlagen

2.1 GPU-Programmierung und Compute Shader

Moderne GPUs sind massiv parallele Prozessoren, die Tausende von Threads gleichzeitig ausführen können. Im Gegensatz zu CPUs, die für sequentielle Aufgaben optimiert sind, eignen sich GPUs ideal für Aufgaben, bei denen **derselbe Algorithmus auf viele Datenpunkte gleichzeitig** angewendet wird (SIMD – Single Instruction, Multiple Data).

Compute Shader sind Programme, die direkt auf der GPU laufen und nicht an die klassische Grafikpipeline (Vertex → Fragment) gebunden sind. Sie arbeiten auf einem flexiblen Grid aus *Threadgroups*:

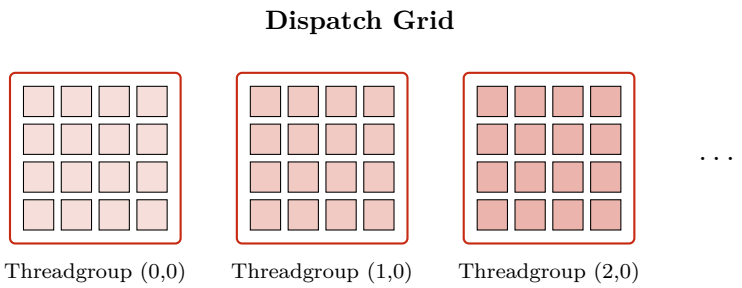

Abbildung 1: Compute-Shader-Dispatch:
Das Bild wird in Threadgroups unterteilt, jeder Thread bearbeitet ein Pixel

Jeder Thread kennt seine Position im Grid (`thread_position_in_grid`) und bearbeitet typischerweise genau ein Pixel. RetroVisor verwendet **16×16 Threadgroups** – jede Gruppe bearbeitet also einen 16×16-Pixel-Block des Ausgabebildes.

2.2 Metal Shading Language (MSL)

MSL ist Apples GPU-Programmiersprache, basierend auf **C++14** mit GPU-spezifischen Erweiterungen.

Zentrale Konzepte:

Konzept	Beschreibung
`kernel`	Einstiegspunkt für Compute Shader (äquivalent zu `main()` in GLSL)
`texture2d<T>`	2D-Textur mit Typparameter (z. B. `float` oder `half`)
`sampler`	Konfiguriert Textur-Sampling (Filter, Adressierung)
`float2/3/4`	Vektortypen (äquivalent zu GLSL `vec2/3/4`)
`constant T&`	Uniform-Daten (read-only, von der CPU übergeben)
`[[ buffer(n) ]]`	Attribut: bindet Parameter an Buffer-Slot n
`[[ texture(n) ]]`	Attribut: bindet Textur an Slot n
`thread_position_in_grid`	Globale Thread-ID (welches Pixel bearbeitet wird)

Tabelle 1: Zentrale MSL-Konzepte

2.3 GLSL vs. MSL – Wesentliche Unterschiede

Die Portierung von GLSL nach MSL erfordert systematische Übersetzungen:

Aspekt	GLSL / Slang	Metal (MSL)
Vektortypen	`vec2`, `vec3`, `vec4`	`float2`, `float3`, `float4`
Textur-Zugriff	`texture(sampler2D, uv)`	`tex.sample(sam, uv)`
Textur + Sampler	Kombiniert (`sampler2D`)	Getrennt (`texture2d` + `sampler`)
Uniforms	`uniform float x;`	`constant Uniforms &u` `[[ buffer(0) ]]`
Fragment-Koord.	`gl_FragCoord`	`thread_position_in_grid`
Shader-Typ	Fragment Shader	Compute Kernel
Modulo	`mod(x, y)`	`x - y * trunc(x/y)` (manuell)
Ausgabe	`FragColor = ...`	`outTexture.write(color, gid)`

Tabelle 2: GLSL-zu-MSL-Übersetzungstabelle

Ein besonders wichtiger Unterschied: GLSL-Shader nutzen die **Fragment-Shader-Pipeline** (ein Dreieck wird gerastert, pro Pixel läuft der Shader). RetroVisor hingegen setzt auf **Compute Shader**, bei denen das Dispatch-Grid explizit berechnet wird. Dies bietet mehr Flexibilität, erfordert aber manuelle Bounds-Checks.

3 Ist-Analyse: CRT-Royale

3.1 Überblick

CRT-Royale ist ein Open-Source-Shader (GPL v2+) von *TroggleMonkey*,
der in der **libretro/slang-shaders**-Sammlung gepflegt wird.
Er gilt als einer der physikalisch genauesten CRT-Simulatoren
und modelliert folgende optische Phänomene:

- **Gamma-Linearisierung:** Umrechnung vom CRT-Gamma-Farbraum in lineares Licht
- **Scanlines:** Horizontale Helligkeitsmodulation (Zeilen eines Kathodenstrahlröhren-Bildes)
- **Phosphormasken:** RGB-Subpixel-Muster (Aperture Grille, Slot Mask, Shadow Mask)
- **Bloom / Halation:** Lichtüberstrahlung durch die Glasscheibe des Monitors
- **Konvergenz:** Leichte Verschiebung der RGB-Elektronenstrahlen
- **Interlacing:** Halbbilder-Simulation (240p/480i)
- **Geometrieverzerrung:** Gewölbte Bildfläche eines CRT-Monitors

3.2 Die 13-Pass-Pipeline

CRT-Royale verarbeitet das Eingabebild in 13 aufeinanderfolgenden Passes. Jeder Pass liest die Ausgabe des vorherigen und schreibt in eine Zwischentextur:

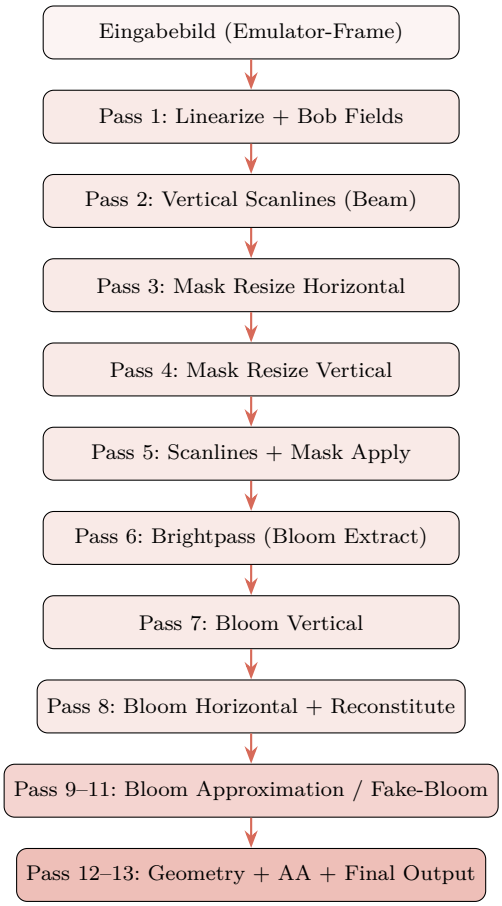

Abbildung 2: CRT-Royale 13-Pass-Pipeline – Datenfluss vom Eingabebild zum fertigen CRT-Effekt

3.3 Quellstruktur

Die relevanten Quelldateien befinden sich unter `crt/shaders/crt-royale/src/`:

Datei	Funktion
`bind-shader-params.h`	Alle ∼45 konfigurierbaren Parameter
`gamma-management.h`	Gamma-Linearisierung und -Encoding
`scanline-functions.h`	Beam-Profile, Scanline-Breite, Interlace-Erkennung
`phosphor-mask-resizing.h`	Lanczos-Sinc-Resampling der Phosphormaske
`bloom-functions.h`	Bloom-Berechnung und -Extraktion
`geometry-functions.h`	Bildschirmkrümmung, Pin-Cushion-Verzerrung
`tex2Dantialias.h`	Anti-Aliasing-Algorithmen (∼80 KB)
`derived-settings-and-constants.h`	Berechnete Konstanten aus User-Settings
`user-settings.h`	Nutzer-Konfiguration und GPU-Capability-Flags

Tabelle 3: Wichtige Header-Dateien des CRT-Royale-Originals

3.4 Konfigurierbare Parameter (Auszug)

CRT-Royale bietet ca. 45 Parameter, die das Erscheinungsbild steuern. Die wichtigsten:

Parameter	Default	Beschreibung
`crt_gamma`	2.5	Gamma des simulierten CRT-Monitors
`lcd_gamma`	2.2	Gamma des realen Ausgabedisplays
`mask_type`	1	Phosphormasken-Typ (0=Grille, 1=Slot, 2=Shadow)
`interlace_detect`	1	Automatische Interlace-Erkennung
`beam_min_sigma`	0.02	Minimale Scanline-Breite (Beam)
`beam_max_sigma`	0.30	Maximale Scanline-Breite (Beam)
`bloom_underestimate`	0.80	Bloom-Intensität
`geom_mode`	0	Geometrie-Modus (0=flat, 1–3=curved)
`geom_radius`	2.0	Krümmungsradius des simulierten CRT
`convergence_offset`	0.0	RGB-Konvergenz-Verschiebung

Tabelle 4: Wichtige CRT-Royale-Parameter

4 Zielarchitektur: RetroVisor

4.1 Überblick

RetroVisor ist eine macOS-Anwendung von Prof. Dr. Hoffmann, die GPU-basierte Bildeffekte als Echtzeit-Overlay auf beliebige Bildschirmfenster anwendet.
Die Anwendung nutzt Apples **ScreenCaptureKit** zur Fensteraufnahme und die **Metal API** für die GPU-beschleunigte Bildverarbeitung.

RetroVisor enthält bereits zwei integrierte Shader:

- **Sankara:** Multi-Pass-Effekt mit Split/Composite/CRT/DotMask-Kernels
- **CRT-Easy:** Einfacher Single-Pass-CRT-Effekt

CRT-Royale MSL wird als **dritter Shader** in RetroVisor integriert.

4.2 Shader-Integrationsmuster

Jeder Shader in RetroVisor besteht aus zwei Dateien:

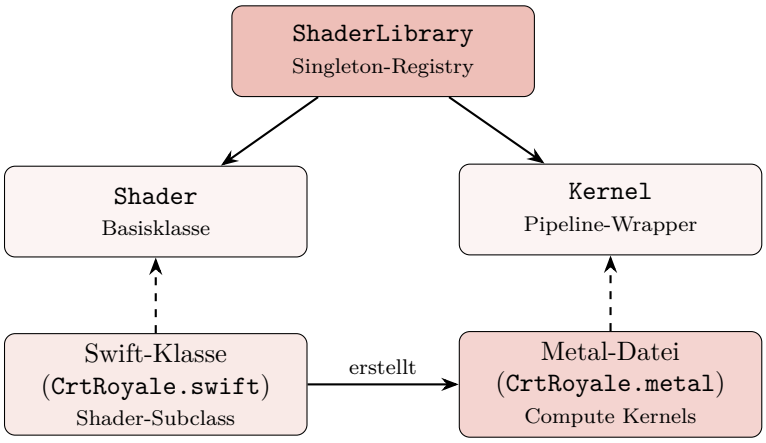

Abbildung 3: RetroVisor Shader-Integrationsmuster

1. **Swift-Klasse** (erbt von `Shader`):
   - Definiert `Uniforms`-Struct (gespiegelt zum Metal-Pendant)
   - Erstellt `Kernel`-Objekte für jeden Pass
   - Implementiert `apply()` zur Orchestrierung der Multi-Pass-Pipeline
   - Definiert UI-Settings (`ShaderSetting`) für Nutzerinteraktion

2. **Metal-Datei** (GPU-Code):
   - Enthält Compute-Kernel-Funktionen (`kernel void`)
   - Liest Eingabetexturen, schreibt Ausgabetexturen
   - Empfängt Uniforms über `constant`-Buffer

4.3 Render-Pipeline

Für jeden Frame durchläuft RetroVisor folgenden Ablauf:

1. **ScreenCaptureKit** liefert ein neues Frame des erfassten Fensters

2. Die `apply()`-Methode des aktiven Shaders wird aufgerufen

3. Für jeden Pass: ein `Kernel.apply()`-Aufruf dispatcht den Compute Shader

4. Jeder Kernel liest von einer Quell-Textur und schreibt in eine Ziel-Textur

5. Das finale Ergebnis wird als Overlay über das Originalfenster gelegt

Bei CRT-Royale bedeutet dies: **13 Kernel-Dispatches pro Frame**, mit 12 Zwischentexturen.

5 Technologieauswahl

Komponente	Technologie	Begründung
GPU-Sprache	Metal Shading Language	Einzige native GPU-API auf macOS; direkte Integration in RetroVisor
Host-Sprache	Swift	RetroVisor ist in Swift geschrieben; Metal-Framework nativ verfügbar
Build-System	Xcode	Standard für macOS-Entwicklung; Metal-Compiler integriert
Quellformat	Slang / GLSL	CRT-Royale-Originalformat in der libretro-Shader-Sammlung
Versionskontrolle	Git / GitHub	Kollaboration und Nachvollziehbarkeit
Testquelle	OpenEmu + 240p Test Suite	NES-Emulator mit standardisierten Testmustern für Shader-Validierung
Referenz	RetroArch (GLSL)	Originaldarstellung zum visuellen Vergleich

Tabelle 5: Technologieübersicht

6 Portierungsstrategie

6.1 Vorgehen

Die Portierung erfolgt **inkrementell, pass-weise**:

1. Jeden Pass einzeln von Slang/GLSL nach MSL übersetzen
2. In RetroVisor integrieren und mit Zwischentexturen verknüpfen
3. Visuell gegen das GLSL-Original validieren (Comparison-Images)
4. Erst nach erfolgreicher Validierung zum nächsten Pass übergehen

Diese Strategie ermöglicht frühes Feedback: bereits nach Pass 1 (Linearisierung) ist ein sichtbarer Effekt vorhanden und testbar.

6.2 Übersetzungsregeln

Folgende systematische Transformationen werden bei jeder Pass-Portierung angewendet:

1. **Vektortypen umbenennen:** `vec2` → `float2`, `vec3` → `float3`, etc.
2. **Textur-Zugriff trennen:** `texture(tex, uv)` → `tex.sample(sam, uv)`
3. **Uniforms bündeln:** Einzelne `uniform`-Deklarationen → `constant Uniforms&`
4. **Fragment → Compute:** `gl_FragCoord` → `thread_position_in_grid`
5. **Ausgabe umschreiben:** `FragColor = ...` → `outTex.write(color, gid)`
6. **Bounds-Check einfügen:** Compute Shader müssen prüfen, ob `gid` innerhalb der Textur liegt
7. **GLSL-Funktionen ersetzen:** `mod()` manuell, `fract()`/`clamp()`/`mix()` sind identisch

6.3 Herausforderungen

- **Multi-Pass-Orchestrierung:** Im Original steuert eine `.glslp`-Preset-Datei die Verkettung. In RetroVisor muss die `apply()`-Methode dies manuell übernehmen.
- **Texture-Size-Semantik:** GLSL-Presets definieren `scale_type` (source/viewport/absolute) pro Pass. Diese Müsse in Swift korrekt berechnet werden.
- **Phosphormasken-Texturen:** PNG-Tile-Texturen müssen als `MTLTexture` geladen und korrekt gesampelt werden.
- **`tex2Dantialias.h`:** Die größte Header-Datei (∼80 KB) mit komplexen Anti-Aliasing-Algorithmen.
- **Präzisionsunterschiede:** GLSL verwendet standardmäßig `mediump`/`highp`; MSL bietet `half` und `float`. Numerische Abweichungen müssen erkannt und behandelt werden.

7 Implementierung

7.1 Aktueller Stand

Zum aktuellen Zeitpunkt ist **Pass 1 (Linearize + Bob Fields)** vollständig portiert, und in RetroVisor integriert.

Der Pass umfasst:

- **Gamma-Linearisierung:**

  `pow(color.rgb, crt_gamma)` – Umrechnung vom nicht-linearen CRT-Farbraum in lineares Licht

- **Interlace-Erkennung:**

  Automatische Erkennung von 480i-Signalen anhand der Texturhöhe (> 288 Zeilen)

- **Bob-Field-Deinterlacing:**

  Bei Interlace-Content wird abwechselnd das gerade/ungerade Halbbild angezeigt

- **Final-Encode:**

  Rück-Übersetzung von linearem Licht in den LCD-Gamma-Farbraum

7.2 Pass 1: Code-Beispiel (MSL)

```
kernel void pass1_linearize(
    texture2d<float, access::read>  inTex   [[ texture(0) ]],
    texture2d<float, access::write> outTex  [[ texture(1) ]],
    constant Uniforms &u            [[ buffer(0) ]],
    uint2 gid                       [[ thread_position_in_grid ]])
{
    if (gid.x >= outTex.get_width() ||
        gid.y >= outTex.get_height()) return;

    float2 uv = float2(gid) / float2(outTex.get_width(),
                                     outTex.get_height());
    float4 color = inTex.read(gid);

    // Gamma-Linearisierung: CRT gamma -> linear
    color.rgb = pow(color.rgb, float3(u.crt_gamma));

    // Interlace bobbing (wenn erkannt)
    if (is_interlaced(u.texture_size.y, u.interlace_detect)) {
        // Bob-Field-Interpolation...
    }

    outTex.write(color, gid);
}
```

Listing 1: Pass 1 – Linearize-Kernel (vereinfacht)

7.3 Swift-Integration

```
@MainActor
final class CrtRoyale: Shader {
    struct Uniforms {
        var crt_gamma: Float = 2.5
        var lcd_gamma: Float = 2.2
        var interlace_detect: UInt32 = 1
        // ... weitere Parameter
    }

    var linearizeKernel: Kernel!
    var finalEncodeKernel: Kernel!
    var linearized: MTLTexture!  // Zwischentextur

    override func apply(commandBuffer: MTLCommandBuffer,
                        in: MTLTexture, out: MTLTexture,
                        rect: CGRect) {
        // Pass 1: Linearize
        linearizeKernel.apply(commandBuffer,
                              source: input, target: linearized)
        // ... weitere Passes ...
        // Final: Encode fuer LCD
        finalEncodeKernel.apply(commandBuffer,
                                source: linearized, target: output)
    }
}
```

Listing 2: CrtRoyale.swift – Multi-Pass-Orchestrierung (vereinfacht)

7.4 Dateien im Projekt

Datei	Beschreibung
`RetroVisor/GPU/CrtRoyale.metal`	Metal Compute Kernels (GPU-seitig)
`RetroVisor/Shaders/CrtRoyale.swift`	Swift-Integrationsklasse (CPU-seitig)
`RetroVisor/Shaders/ShaderLibrary.swift`	Registrierung als 3. Shader
`crt-royale-msl/src/CrtRoyalePass1Linearize.metal`	Standalone-Port in eigenem Repo

Tabelle 6: Projektdateien

8 Validierungskonzept

8.1 Methodik

Die Validierung erfolgt durch **visuellen A/B-Vergleich** zwischen dem GLSL-Original (in RetroArch) und der MSL-Portierung (in RetroVisor):

1. Identisches Eingabebild verwenden (240p Test Suite, definierte Testmuster)
2. Identische Parameter-Einstellungen (crt_gamma, lcd_gamma, mask_type, etc.)
3. Screenshots beider Ausgaben erstellen
4. Pixel-für-Pixel-Differenzbild berechnen
5. Akzeptanzkriterium: $\Delta E < 2.0$ (kaum sichtbarer Unterschied)

8.2 Testmuster

Die **240p Test Suite** (NES-ROM) bietet standardisierte Testbilder:

Testmuster	Validiert
Color Bars	Farbgenauigkeit der Gamma-Transformation
Grid Pattern	Scanline-Alignment und Geometrie
Gradient Ramps	Banding-Artefakte bei Gamma-Konvertierung
Checkerboard	Phosphormasken-Alignment
Solid White	Bloom/Halation-Verhalten
Scrolling Content	Interlace-Bobbing und zeitliche Stabilität

Tabelle 7: Testmuster und validierte Aspekte

9 Evaluierungsplan

9.1 Qualitätsvergleich

- **SSIM** (Structural Similarity Index): Quantitative Ähnlichkeit zum Original
- **ΔE-Farbdifferenz**: CIE-Farbraumdifferenz pro Pixel
- **Subjektive Bewertung**: Sind die typischen CRT-Merkmale visuell korrekt?

9.2 Performancebewertung

- **Frame-Time:** Dauer eines kompletten 13-Pass-Durchlaufs (Ziel: < 16.67 ms für 60 FPS)
- **GPU-Auslastung:** Metal Performance Shaders Profiler in Xcode
- **Speicherverbrauch:** Anzahl und Größe der Zwischentexturen
- **Vergleich:** Gegenüberstellung mit CRT-Easy (Single-Pass) als Baseline

10 Projektplanung

10.1 Phasenübersicht

Phase	Bezeichnung	Inhalt	Status
1	Portierung	Übersetzung aller 13 Passes GLSL → MSL	In Arbeit
2	Integration	Einbindung in RetroVisor, UI-Settings, Texturen	Begonnen
3	Validierung	A/B-Vergleich mit RetroArch, Comparison-Images	Ausstehend
4	Evaluierung	SSIM, ΔE, Frame-Time, GPU-Profiling	Ausstehend

Tabelle 8: Phasenübersicht

10.2 Detailplan Phase 1: Portierung

Pass	Beschreibung	Status
1	Linearize + Bob Fields	✓ Abgeschlossen
2	Vertical Scanlines (Beam)	Ausstehend
3	Mask Resize Horizontal	Ausstehend
4	Mask Resize Vertical	Ausstehend
5	Scanlines + Mask Apply	Ausstehend
6	Brightpass (Bloom Extract)	Ausstehend
7	Bloom Vertical	Ausstehend
8	Bloom Horizontal + Reconstitute	Ausstehend
9	Bloom Approximation	Ausstehend
10	Fake-Bloom Variant A	Ausstehend
11	Fake-Bloom Variant B	Ausstehend
12	Geometry + AA	Ausstehend
13	Final Output	Ausstehend

Tabelle 9: Pass-Portierungsstatus

11 Muss- und Kann-Kriterien

Muss-Kriterien:

M1 Vollständige Portierung aller 13 Passes nach MSL.

M2 Lauffähige Integration in RetroVisor als wählbarer Shader.

M3 Echtzeit-Fähigkeit: ≥ 30 FPS bei 1080p-Eingabe.

M4 Visuell korrekte Darstellung der CRT-Kerneffekte (Scanlines, Maske, Gamma).

M5 Dokumentierte Vergleichsbilder (GLSL-Original vs. MSL-Port).

M6 Quellcode unter Open-Source-Lizenz auf GitHub verfügbar.

Kann-Kriterien:

K1 ≥ 60 FPS bei 1080p (optimiert für Apple Silicon).

K2 Alle $\sim$45 Parameter über RetroVisor-UI konfigurierbar.

K3 SSIM > 0.95 gegenüber dem GLSL-Original.

K4 Unterstützung aller drei Phosphormasken-Typen.

K5 `half`-Präzisions-Variante für reduzierte GPU-Last.

K6 Performance-Vergleich mit CRT-Easy als Baseline.

12 Glossar

Begriff	Definition
CRT	Cathode Ray Tube – Röhrenbildschirm
MSL	Metal Shading Language – Apples GPU-Programmiersprache
GLSL	OpenGL Shading Language – plattformübergreifende GPU-Sprache
Slang	Shader-Format der libretro-Sammlung (GLSL-basiert)
Compute Shader	GPU-Programm ohne Bindung an die Grafikpipeline
Kernel	Metal-Bezeichnung für eine Compute-Shader-Funktion
Threadgroup	Gruppe von GPU-Threads, die gemeinsam ausgeführt werden
Scanline	Horizontale Bildzeile eines CRT-Monitors
Phosphormaske	RGB-Subpixel-Anordnung auf der CRT-Bildschirmfläche
Bloom	Lichtüberstrahlung (Glow) um helle Bildbereiche
Halation	Lichtstreuung durch die Glasscheibe des CRT-Monitors
Interlacing	Halbbildverfahren (gerade/ungerade Zeilen alternierend)
Bob Deinterlacing	Deinterlacing durch abwechselnde Halbbild-Anzeige
Gamma	Nichtlineare Helligkeitskurve (Helligkeit = Signal$^{\gamma}$)
Konvergenz	Ausrichtung der drei Elektronenstrahlen (R, G, B)
Lanczos	Hochwertiges Resampling-Verfahren (Sinc-basiert)
SSIM	Structural Similarity Index – Qualitätsmetrik für Bilder
ΔE	Wahrnehmungsbasierte Farbdifferenz (CIE-Farbraum)
RetroVisor	macOS-App für GPU-basierte Echtzeit-Bildeffekte
libretro	API-Standard für Emulator-Frontends (RetroArch)

Tabelle 10: Glossar